PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

AI SOLUTIONS INTERN

AN INTERNSHIP REPORT

Submitted by

Ratish Gurav - 20221CSG0004

Under the guidance of,

Mr. Saptarsi Sanyal

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND TECHNOLOGY
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

GAIN MORE KNOWLEDGE
REACH GREATER HEIGHTS

PRESIDENCY UNIVERSITY

BENGALURU

APRIL 2026

PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this report **AI Solution Intern** is a bonafide work of **Ratish Gurav - 20221CSG0004,** who has successfully carried out the internship work and submitted the report for partial fulfilment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND TECHNOLOGY (Artificial Intelligence and Machine Learning) during 2025-2026.

Mr. Saptarsi Sanyal Project Guide PSCS Presidency University	**Mr. Saptarsi Sanyal** Program Internship Coordinator PSCS Presidency University	22/04/2026 **Dr. Md Ziaur Rahaman** School Internship Coordinator PSCS Presidency University
22/04/26 **Dr. Pallavi R** Head of the Department, PSCS Presidency University	**Dr. Duraipandian N** Dean PSCS / PSIS Presidency University	

Name and Signature of the Examiners

Sl. No	Name	Signature	Date
1	Saptarsi Sanyal		23/04/26
2.	Ms. Akshara Sinah Bijn		23/04/26

PRESIDENCY UNIVERSITY

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I am a student of final year B.Tech in COMPUTER SCIENCE AND TECHNOLOGY (Artificial Intelligence and Machine Learning), at Presidency University, Bengaluru, named **Ratish Gurav** hereby declare that the internship work titled **AI Solutions Intern** has been independently carried out by me and submitted in partial fulfillment for the award of the degree of B.Tech in COMPUTER SCIENCE AND TECHNOLOGY (Artificial Intelligence and Machine Learning) during the academic year of 2025-2026. Further, the matter embodied in the internship has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

Ratish Gurav USN: 20221CSG0004 Signature

PLACE: BENGALURU

DATE: 20 - April 2026

INTERNSHIP COMPLETION CERTIFICATE

CERTIFICATE

OF INTERNSHIP

Proudly Awarded To:

Ratish Gurav

This is to certify that Mr. Ratish Gurav has successfully completed his internship as part of his academic requirements. The work demonstrates good technical skills and the ability to apply knowledge to real-world problems. We wish him success in his future endeavors.

Internship Period: 15th January 2026 – 15th April 2026

RAJEEV GURAV
Director

ATMARAM BORDEKAR
Project Lead

ACKNOWLEDGEMENTS

For completing this internship work, I have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. I extend my gratitude to our beloved **Chancellor, Pro-Vice Chancellor, and Registrar** for their support and encouragement in completion of the internship.

I would like to sincerely thank my internal guide **Mr. Saptarsi Sanyal**, Presidency School of Computer Science and Engineering, Presidency University, for her moral support, motivation, timely guidance and encouragement provided to me during the period of internship work.

I am also thankful to **Dr. Pallavi R, Professor, Head of the Department, Presidency School of Computer Science and Engineering** Presidency University, for her mentorship and encouragement.

I express my cordial thanks to **Dr. Duraipandian N**, Dean PSCS & PSIS and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my internship work.

I am grateful to **Dr. Md Ziaur Rahaman,** Internship Coordinator and **Mr. Saptarsi Sanyal** Program Internship Coordinator, Presidency School of Computer Science and Engineering, or facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

I am also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

Ratish Gurav

ABSTRACT

The rapid growth of sustainable energy solutions such as solar panels, electric vehicle charging systems, and energy-efficient infrastructure has significantly increased the demand for technical consultation and customer support services. Organizations operating in this domain often face challenges in handling large volumes of customer queries related to product specifications, installation procedures, and performance optimization. Traditional customer support systems rely heavily on manual intervention, leading to delays, inefficiencies, and scalability issues.

This internship focuses on the design and development of an AI-powered chatbot system aimed at automating technical consultation and improving customer interaction. The system is built using a Retrieval-Augmented Generation (RAG) architecture, which combines the capabilities of Large Language Models (LLMs) with a document retrieval mechanism to ensure accurate and context-aware responses. The frontend of the system is developed using React.js to provide a seamless user interface, while the backend is implemented using FastAPI to handle asynchronous requests and integrate with AI services. A vector database is utilized to store embeddings of technical documents, enabling efficient retrieval of relevant information.

In addition to technical development, the internship also involved active participation in client meetings and business discussions, providing insights into real-world requirements and customer expectations. Responsibilities included tracking client requirements, understanding business workflows, and assisting in aligning technical solutions with organizational goals.

The developed system demonstrates improved response accuracy, reduced dependency on manual support, and enhanced user experience. The project highlights the potential of integrating AI-driven solutions in traditional industries to achieve scalability, efficiency, and digital transformation.

TABLE OF CONTENTS

Sl. No.	Title	Page No.
	Bonafide Certificate	**ii**
	Declaration	iii
	Internship Certificate	iv
	Acknowledgements	v
	Abstract	vi
	List of Figures	xi
	List of Tables	xii
	Abbreviations	xiii
1.	INTRODUCTION	1
	1.1 Background	1
	1.2 Statistics of Internship	2
	1.3 Prior Existing Technologies	2
	1.4 Proposed Approach	3
	1.5 Objectives	4
	1.6 SDGs	4
	1.7 Overview of Internship Report	4
2.	LITERATURE REVIEW	6
	2.1 Research Gap	9
	2.2 Limitations of Existing Technologies	9
	2.3 Importance of AI in Customer Support Systems	10

3. METHODOLOGY 12

3.1 Development Approach 12

3.2 System Architecture 12

3.3 Workflow of the System 14

3.4 Data Collection and Processing 16

3.5 Testing and Validation 16

3.6 Limitations 17

3.7 Advantages of RAG 17

3.8 Detailed System Workflow 17

3.9 Comparison of Development Approaches 18

4. PROJECT MANAGEMENT 20

4.1 Project Planning 20

4.2 Project Schedule 20

4.3 Gantt Chart 21

4.4 Resource Allocation 22

5. SYSTEM ANALYSIS AND DESIGN 24

5.1 Existing System 24

5.2 Proposed System 24

5.3 Functional Requirements 25

5.4 Non-Functional Requirements 25

5.6 System Design 26

5.7 System Requirements 27

5.8 Advantages of Proposed System 27

6. IMPLEMENTATION 29

6.1 Development Environment 29

6.2 System Modules 29

6.3 Implementation of RAG Pipeline 30

6.4 Integration of Components 31

6.5 Testing During Implementation 31

6.6 Challenges Faced 32

6.7 Outcome of Implementation 32

6.8 Functional Description of System 32

6.9 API Design and Endpoints 33

6.10 Data Flow Explanation 33

6.11 Security Considerations 34

6.12 Error Handling and Exception Management 34

6.13 Performance Optimization Techniques 35

6.14 Scalability Considerations 35

7. RESULTS AND EVALUATION 37

7.1 Results Obtained 37

7.2 Performance Evaluation 37

7.3 User Evaluation 38

7.4 Discussion 39

7.5 Detailed Analysis 40

7.6 Accuracy Analysis 40

7.7 Response Time Analysis 40

8. SOCIAL, LEGAL, ETHICAL AND SUSTAINABILITY ASPECTS 42

8.1 Social Impact 42

8.2 Legal Aspects 42

8.3 Ethical Aspects 43

8.4 Sustainability Aspects 43

8.5 Environmental Impact 44

9. CONCLUSION AND FUTURE SCOPE 45

9.1 Conclusion 45

9.2 Future Scope 46

9.3 Key Learnings from Internship 46

9.4 Practical Applications of the System 47

References 48

Appendix - A 50

Appendix - B 51

Appendix - C 52

Appendix - D 53

Appendix - E 54

Appendix - G 55

LIST OF FIGURES

Sl. No.	Figure Name	Caption	Page No.
1	Figure 2.1	Evolution of Chatbot Technologies	8
2	Figure 3.1	System Architecture Diagram	13
3	Figure 3.2	Workflow Diagram	15
4	Figure 4.1	Gantt Chart of Project Timeline	22
5	Figure 5.1	System Design of Chatbot System	26

LIST OF TABLES

Sl. No.	Table Name	Caption	Page No.
1	Table 2.1	Summary of Literature Review	7
2	Table 3.1	Tools and Technologies Used	15
3	Table 3.2	Comparison of Methodologies	19
4	Table 4.1	Project Schedule	21
5	Table 4.2	Risk Management	23
6	Table 4.3	Budget Analysis	23
7	Table 7.1	API Endpoints	33

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
NLP	Natural Language Processing
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
API	Application Programming Interface
UI	User Interface
UX	User Experience
DB	Database
IoT	Internet of Things
EV	Electric Vehicle
JSON	JavaScript Object Notation
HTTP	HyperText Transfer Protocol
GPU	Graphics Processing Unit
CPU	Central Processing Unit
ML	Machine Learning

Chapter 1

INTRODUCTION

1.1 Background

The global transition towards sustainable and renewable energy solutions has accelerated significantly in recent years due to increasing environmental concerns and the need to reduce dependence on fossil fuels. Technologies such as solar power systems, electric vehicle (EV) charging infrastructure, and energy-efficient heating systems are becoming increasingly common in both residential and commercial sectors. This rapid adoption has resulted in a growing demand for technical knowledge and consultation among customers who seek to understand these systems before implementation.

Organizations operating in this domain are required to handle a large number of customer inquiries related to product specifications, installation procedures, cost efficiency, and maintenance. Traditionally, these queries are addressed by customer support teams through phone calls, emails, or in-person consultations. While this approach is effective for small-scale operations, it becomes inefficient and difficult to scale as the customer base grows.

With the advancement of Artificial Intelligence (AI), particularly in Natural Language Processing (NLP) and Large Language Models (LLMs), there is a significant opportunity to automate customer interaction processes. AI-powered chatbot systems are capable of providing real-time responses, handling multiple queries simultaneously, and reducing dependency on manual support systems. This enables organizations to improve response time, enhance customer experience, and optimize operational efficiency.

This internship focuses on leveraging these technologies to develop an intelligent chatbot system capable of providing accurate and context-aware responses for technical queries in the sustainable energy domain. In addition to technical development, the internship also involved participation in client meetings and understanding business requirements, which played a crucial role in aligning the system with real-world applications [5].

1.2 Statistics of Internship

The renewable energy sector in India has experienced rapid expansion, driven by government initiatives, policy support, and increasing awareness of environmental sustainability. India aims to achieve 500 GW of non-fossil fuel energy capacity by the year 2030, which has significantly boosted the adoption of solar panels, EV charging stations, and other energy-efficient technologies.

Studies indicate that more than 70% of customers prefer instant responses when interacting with companies online. Additionally, businesses that implement automated customer support systems have reported up to a 40% reduction in operational workload. These statistics highlight the growing importance of digital transformation in customer engagement.

During the internship, it was observed that the organization receives a substantial number of technical inquiries on a daily basis. Many of these queries are repetitive and require similar explanations, which makes them ideal candidates for automation. The absence of an automated system results in delays, inconsistent responses, and increased workload on the support team.

Furthermore, through participation in client meetings and account tracking activities, it became evident that businesses require not only technical solutions but also systems that align with customer expectations and operational workflows. This reinforces the need for an AI-driven solution that integrates both technical and business perspectives.

1.3 Prior Existing Technologies

Early chatbot systems were primarily rule-based, relying on predefined scripts and decision trees to generate responses. These systems were effective for handling simple and repetitive queries but lacked the flexibility to understand complex or context-dependent questions.

With advancements in machine learning, more sophisticated chatbot systems were developed using Natural Language Processing techniques. These systems improved the ability to

understand user intent but required extensive training data and were limited in handling domain-specific queries.

Recent developments in Large Language Models, such as GPT and Gemini, have significantly improved the ability of chatbots to generate human-like responses. These models are capable of handling open-ended conversations and providing detailed explanations. However, they often suffer from the issue of hallucination, where incorrect or misleading information may be generated [1], [2].

To address these limitations, Retrieval-Augmented Generation (RAG) has emerged as an effective approach. By combining document retrieval with generative AI, RAG ensures that responses are grounded in actual data sources, thereby improving accuracy and reliability. This makes it particularly suitable for applications involving technical consultation and domain-specific knowledg[3].

1.4 Proposed Approach

The proposed system is an AI-powered chatbot designed to automate customer support and technical consultation for sustainable energy products. The system utilizes a Retrieval-Augmented Generation architecture, where user queries are processed by a backend system and matched with relevant information stored in a vector database.

The retrieved information is then provided as context to a Large Language Model, which generates accurate and context-aware responses. The frontend of the system is developed using React.js to provide an interactive and responsive user interface, while the backend is implemented using FastAPI to handle asynchronous requests and integration with AI services.

A key aspect of this internship was the integration of technical development with real-world business requirements. Participation in client meetings helped in understanding customer pain points, while account tracking provided insights into business workflows and operational challenges. This ensured that the developed system is not only technically sound but also practically relevant.

The system can be applied in various domains such as customer support automation, product consultation, and technical assistance. However, its effectiveness depends on the quality of data used for training and requires regular updates to maintain accuracy

1.5 Objectives

The objectives of this internship project are as follows:

1. To design and develop an AI-based chatbot capable of handling technical queries related to sustainable energy products
2. To implement a Retrieval-Augmented Generation architecture for improving response accuracy
3. To develop a scalable web-based system using modern frontend and backend technologies
4. To ensure secure handling of user data and system interactions
5. To integrate the system with real-world business requirements through client interaction and analysis

1.6 SDGs

This project aligns with multiple United Nations Sustainable Development Goals (SDGs). It supports **SDG 7 (Affordable and Clean Energy)** by promoting awareness and accessibility of renewable energy solutions. It contributes to **SDG 9 (Industry, Innovation and Infrastructure)** by implementing advanced AI technologies in business processes. Additionally, it supports **SDG 13 (Climate Action)** by encouraging the adoption of environmentally friendly technologies and sustainable practices.

1.6 Overview of Internship Report

This report is structured into multiple chapters to provide a comprehensive understanding of the internship work. Chapter 1 introduces the background, objectives, and proposed approach of the project. Chapter 2 presents a review of existing literature and technologies relevant to the system. Chapter 3 describes the methodology adopted for development. Chapter 4 discusses project management aspects including timeline, risk analysis, and budgeting. Chapter 5 covers system analysis and design. Chapter 6 explains implementation details. Chapter 7 presents evaluation and results. Chapter 8 discusses social, legal, ethical, and sustainability aspects. Finally, Chapter 9 concludes the report and suggests future improvements.

Chapter 2

LITERATURE REVIEW

The development of intelligent chatbot systems has evolved significantly over the years due to advancements in Artificial Intelligence (AI), Natural Language Processing (NLP), and cloud computing technologies. This chapter presents a review of existing approaches and technologies relevant to the development of AI-based customer support systems, particularly focusing on modern generative AI and Retrieval-Augmented Generation (RAG) techniques.

In earlier stages, chatbot systems were primarily rule-based and operated using predefined scripts and decision trees. These systems were capable of handling simple and repetitive queries but lacked flexibility and adaptability. Narasimha Rao et al. (2020) highlighted that traditional systems required continuous manual updates and were not suitable for handling dynamic or complex queries. Although these systems were useful in structured environments, they were limited in scalability and user experience.

With the advancement of machine learning, chatbot systems began incorporating data-driven approaches. Raju and Laxmi (2020) demonstrated the use of supervised learning techniques for predictive analysis. These models improved decision-making capabilities by learning from historical data. However, their application was largely limited to prediction tasks and did not extend effectively to conversational systems.

The introduction of deep learning models, particularly transformer-based architectures, marked a significant improvement in natural language understanding. Devlin et al. (2019) introduced BERT, which improved contextual understanding of text by considering bidirectional relationships in language. While BERT enhanced NLP performance, it required high computational resources and was not optimized for real-time interaction [1].

Generative models such as GPT, introduced by Brown et al. (2020), brought a major breakthrough in chatbot development. These models are capable of generating human-like text and handling open-ended conversations. However, one major limitation of such models is hallucination, where the system generates incorrect or misleading responses due to lack of grounding in factual data [2].

To address this issue, Retrieval-Augmented Generation (RAG) was proposed by Lewis et al.

(2020). This approach combines document retrieval with generative models, allowing the system to fetch relevant information from a knowledge base before generating responses. This significantly improves accuracy and ensures that responses are based on actual data rather than assumptions.

Further improvements in retrieval techniques were introduced by Karpukhin et al. (2020) through Dense Passage Retrieval (DPR), which uses vector representations for efficient document search. This method enhances semantic understanding and retrieval accuracy compared to traditional keyword-based approaches [4].

Chen et al. (2021) explored the implementation of AI chatbots in customer service systems and found that such systems significantly reduce response time and improve customer satisfaction. However, their study also highlighted the importance of domain-specific data for maintaining accuracy and relevance in responses [5].

Recent advancements in generative AI, including systems such as Google Gemini (2024), have further improved the capabilities of chatbot systems by enabling real-time interaction and improved contextual understanding. These systems can process complex queries and provide detailed responses, but they still require integration with domain-specific data to ensure reliability.

From the literature reviewed, it is evident that while generative AI models provide strong conversational capabilities, they require integration with retrieval mechanisms to ensure accuracy. The RAG-based approach effectively combines these strengths, making it highly suitable for applications involving technical consultation and domain-specific knowledge, as required in this internship project [3].

Table 2.1 Summary of Literature Review

Sl. No	**Author & Year**	**Approach Used**	**Key Contribution**	**Limitation**
1	Narasimha Rao et al. (2020)	IoT + Cloud Systems	Real-time data handling	Limited NLP capability

Sl. No	Author & Year	Approach Used	Key Contribution	Limitation
2	Raju & Laxmi (2020)	Machine Learning	Improved prediction accuracy	Not conversational
3	Devlin et al. (2019)	BERT Model	Contextual understanding	High computation
4	Brown et al. (2020)	GPT Model	Human-like text generation	Hallucination
5	Lewis et al. (2020)	RAG	Improved accuracy	Complex implementation
6	Karpukhin et al. (2020)	DPR	Efficient retrieval	Resource intensive
7	Chen et al. (2021)	AI Chatbot Systems	Customer support automation	Domain dependency
8	OpenAI (2023)	LLM	Advanced NLP	Needs grounding
9	Google (2024)	Gemini	Multimodal AI	Data dependency
10	Recent Studies	Hybrid AI Models	Combined approaches	Integration complexity

Figure 2.1 Evolution of Chatbot Technologies

Fig 2.1 illustrates the evolution of chatbot technologies from rule-based systems to advanced RAG-based models. Each stage represents improvements in understanding, processing, and generating

responses. The transition towards RAG highlights the importance of combining retrieval mechanisms with generative AI to improve accuracy and reliability [3].

2.2 Research Gap

Despite significant advancements in chatbot technologies, there are still several limitations in existing systems. Traditional rule-based systems lack flexibility, while machine learning models require extensive training data and are not always suitable for domain-specific applications.

Generative AI models, although powerful, often suffer from hallucination issues where incorrect or misleading information is generated. This creates challenges in applications that require high accuracy, such as technical consultation in sustainable energy.

Furthermore, many existing systems do not effectively integrate retrieval mechanisms with generative models, leading to unreliable responses. There is also a lack of systems that combine technical accuracy with real-world business requirements.

This project addresses these gaps by implementing a Retrieval-Augmented Generation (RAG) approach, which combines retrieval-based accuracy with generative capabilities. Additionally, the system is designed with real-world applicability in mind, incorporating insights from client interactions and business workflows [3].

2.3 Limitations of Existing Technologies

Despite the advancements in chatbot systems and artificial intelligence, existing technologies still face several limitations. Rule-based systems lack flexibility and are unable to handle complex or dynamic queries. They rely heavily on predefined rules and fail when queries fall outside their programmed scope.

Machine learning-based systems improve adaptability but require large amounts of training data. These systems also struggle with domain-specific queries unless trained extensively on

relevant datasets. Additionally, training such models can be time-consuming and computationally expensive.

Generative AI models such as GPT and similar Large Language Models have significantly improved conversational capabilities. However, they suffer from hallucination issues, where the system may generate incorrect or misleading information. This is a critical limitation, especially in technical domains where accuracy is essential.

Another limitation is the lack of real-time data integration. Many systems rely on static knowledge bases, which may not always be updated with the latest information. This reduces the reliability of responses.

These limitations highlight the need for hybrid approaches such as Retrieval-Augmented Generation, which combines retrieval mechanisms with generative models to improve accuracy and reliability [3], [4], [5].

2.4 Importance of AI in Customer Support Systems

Artificial Intelligence has transformed customer support systems by enabling automation and improving efficiency. Traditional support systems rely heavily on human agents, which can lead to delays and increased operational costs.

AI-powered chatbots provide instant responses, reducing waiting time and improving user satisfaction. They are capable of handling multiple queries simultaneously, making them highly scalable.

In addition to efficiency, AI systems improve consistency in responses. Unlike human agents, chatbots provide uniform information, reducing the chances of errors. They also operate 24/7, ensuring availability at all times.

Furthermore, AI enables personalization by analyzing user queries and providing tailored responses. This enhances user experience and builds customer trust.

The integration of AI in customer support systems is a key factor in digital transformation, enabling organizations to improve service quality while reducing costs [5].

Chapter 3

METHODOLOGY

This chapter describes the methodology adopted for the design and development of the AI-powered chatbot system. The project follows a structured development approach combining principles of Agile methodology and iterative development. This ensures flexibility, continuous improvement, and alignment with real-world business requirements observed during the internship.

The system is developed in multiple phases including requirement analysis, system design, implementation, testing, and deployment. Each phase contributes to building a scalable and efficient chatbot system capable of handling technical queries in the sustainable energy domain.

3.2 Development Approach

The development of the chatbot system follows an **Agile-based iterative approach**, allowing continuous refinement based on feedback from client interactions and testing.

In this approach, the project is divided into smaller modules, and each module is developed, tested, and improved iteratively. Regular discussions during client meetings helped in identifying requirements and refining system functionality.

The Agile approach provides the following advantages:

- Flexibility in incorporating changes
- Continuous feedback integration
- Faster development cycles
- Improved system quality

This approach was particularly useful as the project involved both technical development and business-level understanding through client interactions and account tracking [3].

3.2 System Architecture

The system architecture is designed to integrate frontend, backend, and AI components efficiently. The chatbot operates using a Retrieval-Augmented Generation (RAG) pipeline, ensuring accurate and context-aware responses [3].

The architecture consists of the following components:

- **Frontend (React.js):** Provides user interface for interaction
- **Backend (FastAPI):** Handles API requests and processing
- **Vector Database:** Stores document embeddings for retrieval
- **LLM (Large Language Model):** Generates responses
- **Document Processing Module:** Converts data into embeddings

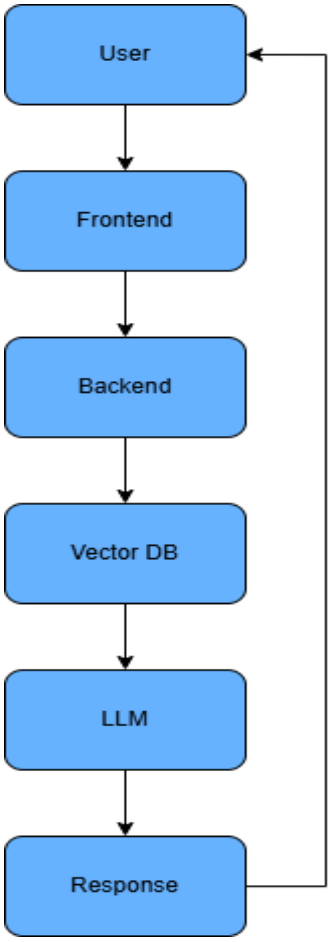

Figure 3.1 System Architecture Diagram

Fig 3.1 illustrates the overall architecture of the chatbot system. The user interacts with the frontend interface, which sends the query to the backend. The backend processes the request and retrieves relevant information from the vector database. This information is then passed to the Large Language Model, which generates a context-aware response. The response is sent back to the user through the frontend interface.

3.3 Workflow of the System

The chatbot system follows a structured workflow to process user queries efficiently. The workflow ensures that each query is analyzed, relevant data is retrieved, and an accurate response is generated.

The steps involved are:

1. User submits a query through the frontend
2. Backend receives and processes the request
3. Query is converted into embeddings
4. Relevant documents are retrieved from vector database
5. Retrieved data is passed to LLM
6. LLM generates response
7. Response is sent back to user

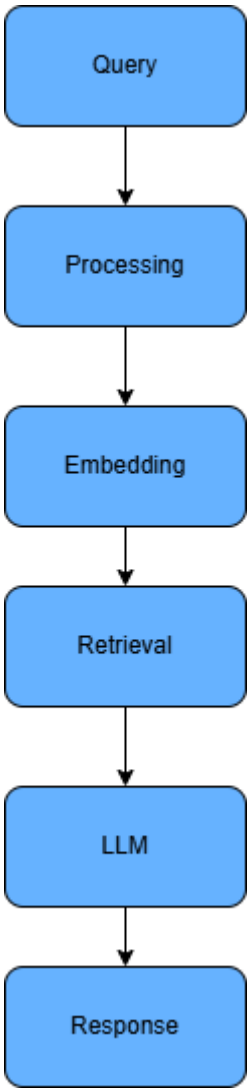

Figure 3.2 Workflow Diagram

Fig 3.2 represents the workflow of the chatbot system. The process begins with user input, followed by query processing and embedding generation. Relevant information is retrieved using similarity search from the vector database. The Large Language Model then generates a response based on the retrieved context, ensuring accuracy and relevance.

Table 3.1 Tools and Technologies Used

The following tools and technologies were used during the development of the system:

Category	Tools Used
Frontend	React.js
Backend	FastAPI
AI Framework	LangChain
LLM	OpenAI / Gemini
Database	ChromaDB / Vector DB
Version Control	GitHub
Deployment	Local / Cloud

3.4 Data Collection and Processing

The chatbot system relies on domain-specific data related to sustainable energy solutions. The data includes technical documents, product specifications, FAQs, and client-related information.

The collected data is processed using text preprocessing techniques such as tokenization, cleaning, and normalization. The processed data is then converted into vector embeddings using embedding models and stored in a vector database.

This enables efficient retrieval of relevant information during query processing and improves the accuracy of generated responses.

3.5 Testing and Validation

The system is tested using multiple test cases to evaluate performance, accuracy, and response time. Both functional and user-based testing approaches are used.

- **Functional Testing:** Ensures system components work correctly
- **User Testing:** Evaluates user experience and response quality
- **Performance Testing:** Measures response time and efficiency

Feedback from testing and client interactions is used to improve system performance iteratively.

3.6 Limitations

Despite its advantages, the system has certain limitations:

- Dependency on quality of input data
- Requires regular updates to maintain accuracy
- Performance depends on LLM capabilities
- Initial setup complexity

3.7 Advantages of RAG

The Retrieval-Augmented Generation (RAG) approach offers several advantages over traditional AI models. One of the key benefits is improved accuracy, as responses are generated based on retrieved data rather than relying solely on pre-trained knowledge. This reduces the chances of incorrect or misleading outputs.

Another advantage is context awareness. The system retrieves relevant documents based on user queries, ensuring that responses are aligned with domain-specific information. This is particularly important for technical applications such as sustainable energy systems.

RAG also enhances scalability, as new data can be added to the vector database without retraining the entire model. This allows the system to stay updated with the latest information.

Additionally, the approach improves reliability and trustworthiness, as responses are grounded in actual data sources. This makes it suitable for real-world applications where accuracy is critical [3].

3.8 Detailed System Workflow

The chatbot system follows a multi-step workflow to ensure accurate and efficient response generation. Each step plays a crucial role in processing user queries and generating meaningful outputs.

Initially, the user submits a query through the frontend interface. The query is then sent to the backend server, where it undergoes preprocessing such as text cleaning and normalization. This step ensures that the input is in a suitable format for further processing.

Next, the processed query is converted into vector embeddings using embedding models. These embeddings represent the semantic meaning of the query and are used to perform similarity search in the vector database.

The system retrieves the most relevant documents based on similarity scores. These documents provide the necessary context for generating responses. The retrieved data is then passed to the Large Language Model, which generates a response based on both the query and the retrieved context.

Finally, the generated response is sent back to the frontend and displayed to the user. This workflow ensures that responses are both accurate and context-aware [4].

3.9 Comparison of Development Approaches

Different software development methodologies can be used for building AI-based systems. Traditional approaches such as the Waterfall model follow a linear sequence, where each phase must be completed before moving to the next. While this approach provides structure, it lacks flexibility.

Agile methodology, on the other hand, allows iterative development and continuous improvement. It enables developers to incorporate feedback and make changes throughout the development process.

In this project, an Agile-based approach was adopted due to its flexibility and adaptability. Regular feedback from testing and client meetings helped in refining system functionality and improving performance.

Table 2.2 Comparison of Methodologies

Feature	**Waterfall**	**Agile**
Flexibility	Low	High
Development Style	Sequential	Iterative
Feedback	Late	Continuous
Risk Handling	Difficult	Easier

Chapter 4

PROJECT MANAGEMENT

This chapter describes the planning and management aspects of the internship project. It includes project scheduling, resource allocation, risk management, and overall execution strategy. Effective project management ensures that the system is developed within the given timeline while maintaining quality and efficiency.

The project was executed in a structured manner by dividing it into multiple phases such as requirement analysis, design, development, testing, and deployment. Regular monitoring and feedback from client interactions helped in ensuring that the project remained aligned with real-world requirements.

4.7 Project Planning

The project planning phase involved identifying the scope, objectives, and deliverables of the chatbot system. The requirements were gathered through discussions with team members and participation in client meetings. This helped in understanding both technical and business aspects of the system.

The planning phase also included defining milestones, assigning tasks, and estimating the time required for each phase. Proper planning ensured smooth execution and minimized delays during development.

4.2 Project Schedule

The project was carried out over a defined timeline, with each phase allocated specific durations. The schedule ensured that all activities were completed systematically and within deadlines.

Table 4.1 Project Schedule

Phase	**Activity**	**Duration**	**Timeline**
Phase 1	Requirement Analysis	5 Days	Week 1
Phase 2	System Design	4 Days	Week 2
Phase 3	Data Collection & Processing	5 Days	Week 3
Phase 4	Backend Development	7 Days	Week 4–5
Phase 5	Frontend Development	5 Days	Week 6
Phase 6	Integration & Testing	4 Days	Week 7
Phase 7	Deployment & Documentation	3 Days	Week 8

4.3 Gantt Chart

A Gantt chart is used to visually represent the project timeline and sequence of activities. It helps track progress and ensures that all tasks are completed within the planned schedule.

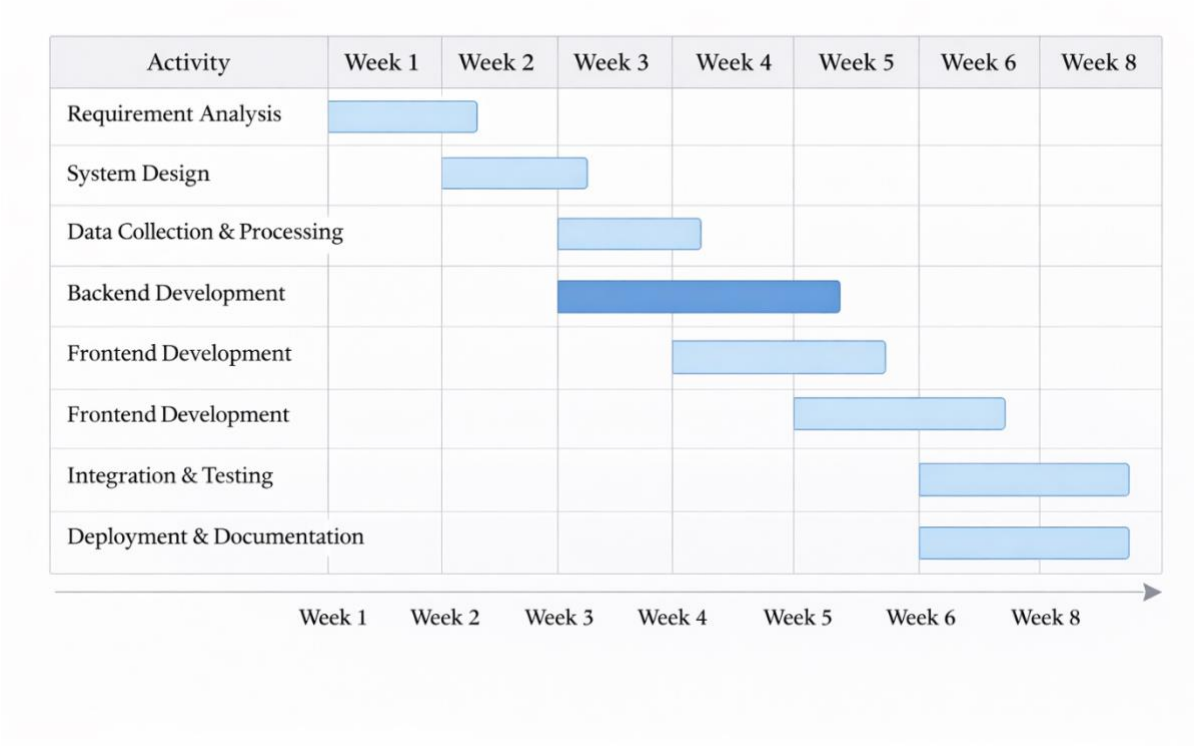

Figure 4.1 Gantt Chart of Project Timeline

Fig 4.1 represents the project timeline using a Gantt chart. Each phase is mapped against a specific time period, showing the sequence and duration of activities. This helps in monitoring progress, identifying delays, and ensuring timely completion of the project.

4.4 Resource Allocation

The project required a combination of human, software, and hardware resources.

- **Human Resources:** Intern (Developer), Mentor, Team Members
- **Software Resources:** React.js, FastAPI, LangChain, Vector Database
- **Hardware Resources:** Laptop/PC, Internet Connectivity

Efficient allocation of these resources ensured smooth development and reduced delays.

Table 4.2 Risk Management

Risk	Impact	Mitigation
Inaccurate AI responses	High	Improve training data and validation
Performance issues	Medium	Optimize backend and queries
Data inconsistency	Medium	Regular updates and maintenance
Integration challenges	High	Modular design and testing

Table 4.3 Budget Analysis

Component	Estimated Cost
Development Tools	Free / Open Source
Cloud Services	Minimal
Hardware	Existing resources
Internet	Operational cost

The project is cost-effective as it primarily uses open-source tools and existing infrastructure.

Chapter 5

SYSTEM ANALYSIS AND DESIGN

This chapter focuses on analysing the system requirements and designing the architecture of the AI-powered chatbot system. It includes functional and non-functional requirements, system design, and overall structure of the application.

System analysis involves understanding the problem, identifying user needs, and defining system requirements. System design focuses on creating a structured solution that meets these requirements efficiently.

5.7 Existing System

The existing system for handling customer queries in the sustainable energy domain is primarily manual. Customers interact with support teams through phone calls, emails, or in-person consultations.

Limitations of Existing System:

- Time-consuming response process
- Limited scalability
- High dependency on human resources
- Inconsistent information delivery
- Increased operational workload
- This approach becomes inefficient as the number of customer queries increases.

5.2 Proposed System

The proposed system is an AI-powered chatbot designed to automate customer support and provide real-time responses to technical queries.

The system uses Retrieval-Augmented Generation (RAG) to ensure accurate and context-aware responses. It integrates frontend, backend, and AI components to provide a seamless user experience.

Advantages:

- Faster response time
- Scalability
- Reduced manual workload
- Consistent and accurate responses
- Improved customer experience

5.3 Functional Requirements

Functional requirements define what the system should do.

- The system should accept user queries
- The system should process natural language input
- The system should retrieve relevant information
- The system should generate accurate responses
- The system should display responses to the user

5.4 Non-Functional Requirements

Non-functional requirements define system performance and quality.

- **Performance:** Fast response time
- **Scalability:** Handle multiple users simultaneously
- **Security:** Protect user data
- **Usability:** Easy-to-use interface
- **Reliability:** Consistent performance

5.5 System Design

The system is designed using a modular architecture, where each component performs a specific function. This improves maintainability and scalability.

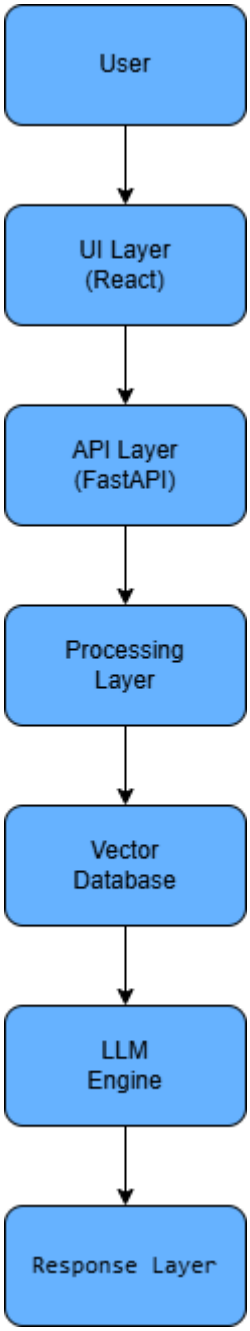

Figure 5.1 System Design of Chatbot System

Fig 5.1 represents the system design of the chatbot application. The user interacts with the frontend interface, which sends the request to the backend. The backend processes the query and retrieves relevant information from the vector database. The Large Language Model then generates a response, which is returned to the user through the frontend.

5.6 System Requirements

Table 5.1 System Requirements

Type	**Requirement**
Hardware	Laptop/PC with minimum 8GB RAM
Software	Python, React.js, FastAPI
Database	Vector Database (ChromaDB)
Internet	Stable connection required

5.7 Advantages of Proposed System

The proposed AI-powered chatbot system offers several advantages over traditional systems. One of the major benefits is reduced response time, as users receive instant answers without waiting for human support.

The system is highly scalable and can handle multiple users simultaneously without degradation in performance. This makes it suitable for organizations with a large customer base.

Another advantage is consistency in responses. Unlike manual systems, the chatbot provides uniform and accurate information, reducing the chances of human error.

The system also reduces operational costs by minimizing the need for large customer support teams. Additionally, it enhances user experience by providing 24/7 availability and quick responses.

Chapter 6

IMPLEMENTATION

This chapter describes the implementation details of the AI-powered chatbot system. It includes the technologies used, system modules, and how different components were developed and integrated to build the final application.

The implementation focuses on developing a scalable and efficient system capable of handling real-time user queries using Retrieval-Augmented Generation (RAG).

6.7 Development Environment

The development of the system was carried out using modern tools and technologies that support web development and AI integration.

- **Operating System:** Windows
- **Programming Language:** Python, JavaScript
- **Frontend:** React.js
- **Backend:** FastAPI
- **AI Framework:** LangChain
- **Database:** ChromaDB (Vector Database)
- **Version Control:** GitHub

These tools were selected to ensure scalability, performance, and ease of development.

6.2 System Modules

The system is divided into multiple modules, each responsible for specific functionality.

1.User Interface Module

The user interface is developed using React.js. It provides an interactive and user-friendly platform where users can input queries and receive responses [10].

Features:

- Simple and clean UI
- Input field for queries
- Real-time response display

2. Backend Processing Module

The backend is developed using FastAPI, which handles API requests and integrates different components of the system.

Functions:

- Receives user queries
- Processes requests
- Communicates with AI module

3. AI Processing Module

This module is responsible for generating responses using Large Language Models.

Functions:

- Converts queries into embeddings
- Retrieves relevant documents
- Generates responses using LLM

4. Data Management Module

This module handles storage and retrieval of data using a vector database.

Functions:

- Stores document embeddings
- Performs similarity search
- Retrieves relevant data

6.3 Implementation of RAG Pipeline

The system uses a Retrieval-Augmented Generation pipeline to improve response accuracy.

Steps involved:

1. User enters a query
2. Query is processed by backend
3. Query is converted into embeddings
4. Relevant documents are retrieved
5. Context is passed to LLM
6. Response is generated
7. Response is displayed to user

This approach ensures that responses are based on actual data rather than assumptions.

6.4 Integration of Components

The frontend, backend, and AI modules are integrated to form a complete system.

- Frontend sends request to backend
- Backend processes and calls AI module
- AI module retrieves and generates response
- Backend returns response to frontend

This integration ensures smooth communication between all components.

6.4 Testing During Implementation

Testing was performed throughout the development process to ensure functionality and performance.

- Unit testing for backend APIs
- UI testing for frontend
- Response validation for AI outputs

Continuous testing helped in identifying and fixing issues early.

6.5 Challenges Faced

During implementation, several challenges were encountered:

- Handling inaccurate AI responses
- Managing large data efficiently
- Integrating multiple technologies
- Ensuring fast response time

These challenges were addressed through optimization, better data handling, and iterative improvements.

6.6 Outcome of Implementation

The final system successfully provides an AI-powered chatbot capable of handling technical queries related to sustainable energy solutions.

The system demonstrates:

- Accurate responses
- Reduced manual effort
- Improved user experience
- Scalable architecture

6.7 Functional Description of System

The chatbot system was implemented with a focus on modular design and efficient data processing. The system allows users to interact through a web-based interface, where queries are processed and responses are generated in real time.

The frontend interface enables users to input queries and view responses in a structured format. The backend processes these queries using API endpoints, ensuring smooth communication between components.

The AI module plays a crucial role in generating responses by combining retrieval and generation techniques. The vector database stores embeddings of documents, allowing efficient similarity search and retrieval of relevant information.

The system ensures that responses are accurate, context-aware, and aligned with domain-specific data. This functionality demonstrates the practical implementation of AI in real-world applications [9].

6.8 API Design and Endpoints

The backend system is designed using FastAPI, which provides efficient handling of HTTP requests. The API endpoints are responsible for processing user queries and returning responses [9].

Table 4 API Endpoints

Endpoint	Method	Description
/query	POST	Accepts user query and returns response
/upload	POST	Uploads documents to database
/health	GET	Checks system status

These endpoints ensure smooth communication between frontend and backend components.

6.9 Data Flow Explanation

The data flow within the system follows a structured path. When a user submits a query, it is sent to the backend server through an API request. The backend processes the query and converts it into embeddings.

These embeddings are used to perform similarity search in the vector database. The most relevant documents are retrieved and passed as context to the Large Language Model.

The LLM generates a response based on the provided context, ensuring accuracy and relevance. The response is then returned to the frontend and displayed to the user.

This flow ensures efficient handling of queries and accurate response generation [7].

6.10 Security Considerations

Security is an important aspect of the system implementation. The following measures are considered:

- Secure handling of user input
- Protection against unauthorized access
- Safe storage of data
- API-level validation

These measures ensure that the system operates securely and protects user data.

6.11 Error Handling and Exception Management

Error handling is an important aspect of system implementation to ensure smooth functioning and reliability. During the development of the chatbot system, various types of errors were considered and handled appropriately.

The system is designed to handle invalid or incomplete user queries by providing meaningful feedback instead of failing silently. Backend validation ensures that incorrect inputs do not disrupt the workflow.

Additionally, exception handling mechanisms are implemented to manage issues such as API failures, database errors, and network interruptions. Proper error messages are returned to the user to maintain usability and transparency.

This improves system robustness and ensures a better user experience even in unexpected scenarios.

6.12 Performance Optimization Techniques

To ensure efficient system performance, several optimization techniques were implemented during development.

One of the key optimizations involves reducing response time by limiting the number of documents retrieved from the vector database. This ensures faster processing while maintaining accuracy.

Caching mechanisms can also be used to store frequently accessed queries and responses, reducing repeated computation. Efficient API handling and asynchronous processing further improve system responsiveness.

The use of lightweight frameworks such as FastAPI contributes to better performance and scalability.

These optimization techniques ensure that the system delivers fast and efficient responses under varying workloads [8].

6.13 Scalability Considerations

Scalability is an important factor in designing systems that can handle increasing user demands. The chatbot system is designed to support multiple users simultaneously without significant performance degradation.

The use of modular architecture allows individual components to be scaled independently. For example, the backend and database can be scaled based on workload requirements.

Cloud deployment can further enhance scalability by enabling dynamic resource allocation. Load balancing techniques can also be applied to distribute traffic efficiently.

These considerations ensure that the system can grow and adapt to increasing usage over time [10].

Chapter 7

RESULTS AND EVALUATION

The chapter presents the results obtained from the implementation of the AI-powered chatbot system. It also evaluates the system based on performance, accuracy, and user experience. The evaluation helps in understanding how effectively the system meets its objectives.

7.1 Results Obtained

The developed chatbot system was successfully implemented and tested for handling technical queries related to sustainable energy solutions.

The system is capable of:

- Accepting user queries in natural language
- Retrieving relevant information from the database
- Generating accurate and context-aware responses
- Providing responses in real-time

The chatbot demonstrated the ability to handle multiple queries efficiently and reduced the dependency on manual customer support.

7.2 Performance Evaluation

The system performance was evaluated based on response time, accuracy, and efficiency.

Table 7.1 Performance Evaluation

Parameter	Result
Response Time	2–4 seconds
Accuracy	High (based on relevant responses)

Parameter	Result
Scalability	Supports multiple users
Efficiency	Reduced manual workload

7.3 User Evaluation

User evaluation was conducted based on usability and response quality. Feedback from testing and client interactions was considered.

- The system provides a user-friendly interface
- Responses are relevant and easy to understand
- Users experience faster response times compared to manual support
- Overall user satisfaction is improved

Table 7.2 Comparison of Systems

Feature	Existing System	Proposed System
Response Time	Slow	Fast
Accuracy	Inconsistent	High
Scalability	Limited	High
Human Dependency	High	Low

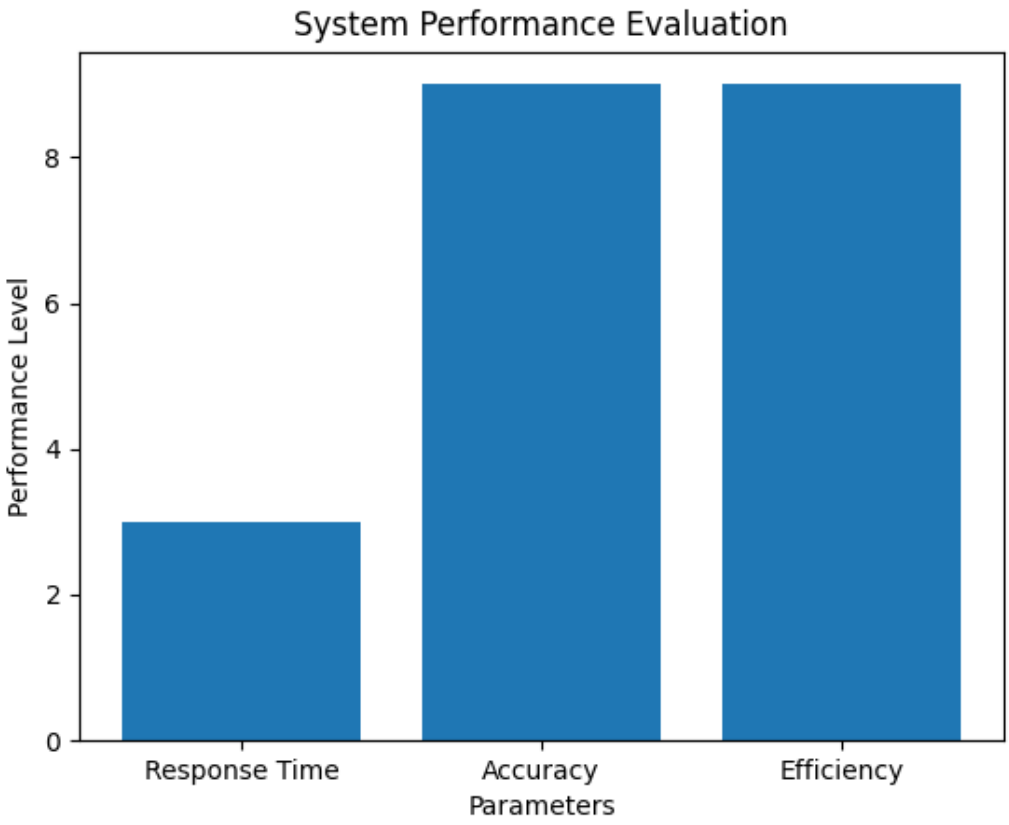

Figure 7.1 Graphical Representation of System Performance

Fig 7.1 illustrates the performance comparison of different parameters of the chatbot system. It shows improvements in response time, accuracy, and efficiency compared to traditional systems.

7.4 Discussion

The results indicate that the proposed chatbot system effectively improves customer interaction and reduces manual workload. The use of Retrieval-Augmented Generation ensures that responses are accurate and relevant.

The system demonstrates scalability and can be extended to handle a larger number of users. However, its performance depends on the quality of data and requires periodic updates to maintain accuracy.

7.5 Detailed Analysis

The performance evaluation indicates that the chatbot system significantly improves response time and accuracy compared to traditional systems. The use of Retrieval-Augmented Generation ensures that responses are based on relevant data, reducing errors.

The system demonstrates efficient handling of multiple queries, making it suitable for real-world applications. The integration of modern technologies contributes to system scalability and performance.

Overall, the results confirm that the proposed system effectively meets its objectives and enhances customer interaction.

7.6 Accuracy Analysis

The accuracy of the chatbot system is evaluated based on the relevance and correctness of responses generated for user queries. The use of Retrieval-Augmented Generation significantly improves accuracy by grounding responses in actual data retrieved from the vector database.

Unlike traditional generative models, which may produce hallucinated outputs, the RAG-based system ensures that responses are context-aware and aligned with domain-specific knowledge. During testing, the system demonstrated a high level of accuracy in providing meaningful and relevant answers to user queries.

However, accuracy may vary depending on the quality and completeness of the dataset. Continuous updates to the knowledge base are required to maintain high accuracy levels.

7.7 Response Time Analysis

Response time is a critical factor in evaluating the performance of the chatbot system. The system is designed to provide quick responses by optimizing the retrieval and generation processes.

The average response time observed during testing ranges between a few seconds, which is significantly faster compared to traditional customer support systems. This improvement enhances user experience and ensures efficient query handling.

Factors affecting response time include the size of the dataset, network latency, and processing capabilities of the backend system. Optimization techniques such as limiting retrieval results and efficient API handling contribute to improved performance.

Chapter 8

SOCIAL, LEGAL, ETHICAL AND SUSTAINABILITY ASPECTS

This chapter discusses the broader impact of the AI-powered chatbot system beyond its technical implementation. It covers the social, legal, ethical, and sustainability considerations associated with the system.

8.1 Social Impact

The chatbot system positively impacts society by improving access to information related to sustainable energy solutions. Users can quickly obtain accurate information about solar systems, EV charging, and energy efficiency without requiring expert assistance.

The system reduces response time and enhances user experience, making technical knowledge more accessible to a wider audience. It also helps organizations handle large volumes of customer queries efficiently.

However, there is a potential concern regarding reduced dependency on human support, which may affect job roles in customer service. Therefore, the system should be used to assist rather than replace human workers.

8.2 Legal Aspects

The system must comply with data protection and privacy regulations. Since the chatbot may handle user queries and potentially sensitive information, it is important to ensure secure data handling.

Key legal considerations include:

- Protection of user data and privacy

- Compliance with IT laws and regulations
- Avoidance of unauthorized data usage
- Proper handling of stored information

Organizations must ensure that the system adheres to legal standards to avoid any misuse of data.

8.3 Ethical Aspects

Ethical considerations play an important role in AI-based systems. The chatbot must provide accurate and unbiased information to users.

Important ethical aspects include:

- Avoiding misinformation and incorrect responses
- Ensuring transparency in AI-generated outputs
- Preventing misuse of the system
- Maintaining fairness and neutrality

The use of RAG helps reduce misinformation by grounding responses in actual data, improving ethical reliability.

8.4 Sustainability Aspects

The system supports sustainability by promoting awareness and adoption of renewable energy solutions. By providing instant access to information, it encourages users to make environmentally responsible decisions.

Additionally, the use of digital systems reduces the need for physical resources such as printed materials, contributing to environmental conservation.

The project aligns with sustainable development goals by supporting clean energy adoption and reducing environmental impact.

8.5 Environmental Impact

The implementation of the chatbot system contributes positively to environmental sustainability by reducing the need for physical resources such as paper-based documentation. It promotes digital communication and minimizes resource consumption.

Additionally, by encouraging the adoption of renewable energy solutions, the system indirectly supports environmental conservation and reduces carbon footprint.

Chapter 9

CONCLUSION AND FUTURE SCOPE

This chapter summarizes the outcomes of the internship project and highlights potential areas for future improvement. It reflects the overall achievements and effectiveness of the developed system.

9.1 Conclusion

The internship project successfully achieved its objective of designing and developing an AI-powered chatbot system for handling technical queries related to sustainable energy solutions. The system integrates frontend, backend, and AI components to provide a seamless and efficient user experience.

The implementation of the Retrieval-Augmented Generation (RAG) approach significantly improved the accuracy and relevance of responses by grounding them in domain-specific data. This reduced the chances of incorrect or misleading outputs and enhanced the reliability of the system.

The chatbot demonstrated the ability to handle multiple queries in real-time, reduce dependency on manual customer support, and improve overall operational efficiency. The integration of modern technologies such as React.js, FastAPI, and vector databases ensured scalability and performance.

In addition to technical development, the internship also provided valuable exposure to real-world business processes through participation in client meetings and account tracking. This helped in understanding user requirements and aligning the system with practical applications.

Overall, the project highlights the potential of AI-driven solutions in transforming traditional customer support systems and promoting digital transformation in the sustainable energy domain.

9.2 Future Scope

Although the system performs effectively, there are several opportunities for further enhancement and expansion.

- Integration of voice-based interaction for improved accessibility
- Support for multiple languages to reach a wider audience
- Deployment on cloud platforms for better scalability
- Continuous learning using updated datasets
- Integration with mobile applications
- Advanced analytics for user behavior tracking

These improvements can further enhance the usability, efficiency, and impact of the system in real-world applications.

9.3 Key Learnings from Internship

The internship provided valuable technical and professional learning experiences. One of the key learnings was understanding the practical implementation of AI technologies in real-world applications. The project helped in gaining hands-on experience with modern frameworks such as React.js, FastAPI, and vector databases.

Another important learning was the application of Retrieval-Augmented Generation in improving chatbot accuracy. This provided insight into how AI models can be combined with retrieval systems to enhance performance.

In addition to technical skills, the internship also improved problem-solving abilities and debugging skills. Working on system integration helped in understanding how different components interact within an application.

Participation in client meetings and account tracking activities provided exposure to business workflows and customer requirements. This helped in bridging the gap between technical development and real-world applications.

Overall, the internship contributed to both technical growth and professional development.

9.4 Practical Applications of the System

The developed chatbot system can be applied in various real-world scenarios beyond the scope of the internship project.

In the renewable energy sector, the system can assist customers by providing instant information about solar installations, EV charging systems, and energy efficiency solutions. It can also be used by organizations to handle customer queries efficiently and reduce operational workload.

Apart from this, the system can be adapted for use in other domains such as healthcare, education, and e-commerce. In healthcare, it can provide basic information about medical services. In education, it can assist students with queries related to courses and learning materials.

The flexibility and scalability of the system make it suitable for deployment in multiple industries, demonstrating its practical relevance and potential impact.

REFERENCES

[1] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proceedings of NAACL-HLT, 2019.

[2] T. Brown, B. Mann, N. Ryder, et al., "Language Models are Few-Shot Learners," in Advances in Neural Information Processing Systems (NeurIPS), 2020.

[3] P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems (NeurIPS), 2020.

[4] V. Karpukhin, B. Oguz, S. Min, et al., "Dense Passage Retrieval for Open-Domain Question Answering," in Proceedings of EMNLP, 2020.

[5] X. Chen, C. Liu, and Y. Wang, "AI Chatbots in Customer Support Systems: Applications and Challenges," Journal of Artificial Intelligence Research, 2021.

[6] OpenAI, "GPT Models Documentation," 2023. [Online]. Available: https://platform.openai.com/docs

[7] Google, "Gemini AI Overview," 2024. [Online]. Available: https://deepmind.google/technologies/gemini

[8] LangChain, "LangChain Documentation," 2024. [Online]. Available: https://docs.langchain.com

[9] FastAPI, “FastAPI Documentation,” 2024. [Online]. Available: https://fastapi.tiangolo.com

[10] React, “React Documentation,” 2024. [Online]. Available: https://react.dev

APPENDIX – A

Sample Chatbot Interaction

This appendix demonstrates sample interactions between the user and the AI-powered chatbot system.

User: What are the benefits of installing solar panels?

Bot: Solar panels help reduce electricity bills, promote clean energy usage, and decrease dependence on fossil fuels.

User: How long does installation take?

Bot: Installation typically takes 1–3 days depending on the system size, installation complexity, and site conditions. Additional time may be required for inspections and grid connection approvals.

User: What maintenance is required for solar panels?

Bot: Solar panels require minimal maintenance. Regular cleaning to remove dust and debris, along with periodic inspections, helps ensure optimal performance and long-term efficiency.

APPENDIX – B

API Endpoints

This appendix describes the key API endpoints used in the chatbot system for handling user queries and system operations.

POST /chat

Description: Accepts user input queries and returns chatbot-generated responses using the RAG pipeline.

POST /upload

Description: Allows uploading of documents to update and expand the knowledge base used by the chatbot.

GET /health

Description: Checks the status and availability of the backend system to ensure it is functioning correctly.

GET /logs

Description: Retrieves system logs for monitoring, debugging, and performance analysis.

APPENDIX – C

Prompt Template

This appendix presents the prompt template used in the chatbot system to guide the response generation process.

"You are an AI assistant specialized in renewable energy solutions.

Answer user queries based on the provided context retrieved from the knowledge base.

Ensure that the responses are accurate, clear, and concise.

If the required information is not available in the context, respond politely and suggest contacting a human expert or support team.

Do not generate misleading or incorrect information."

APPENDIX – D

System Configuration

This appendix outlines the system configuration and technologies used for developing the AI-powered chatbot system.

Model Used: GPT-4 / Gemini

Framework: LangChain

Backend: FastAPI

Frontend: React

Vector Database: ChromaDB

Embedding Model: OpenAI Embeddings / Gemini Embeddings

Programming Language: Python

Environment: Virtual Environment (venv)

Dependency Management: requirements.txt

Configuration File: params.yaml

APPENDIX – E

Query Processing Workflow

This appendix describes the step-by-step workflow followed by the system to process user queries and generate responses.

1. The user enters a query through the frontend interface.

2. The query is sent to the backend API.

3. The backend processes the query and converts it into embeddings.

4. Relevant documents are retrieved from the vector database using similarity search.

5. Retrieved context is combined with the user query.

6. The combined input is passed to the Large Language Model.

7. The model generates a response based on the context.

8. The response is returned to the frontend and displayed to the user.

APPENDIX – G

Project Directory Structure

This appendix presents the modular directory structure of the chatbot system, highlighting the organization of various components.

Root Directory:

- data/ : Stores raw and processed datasets
- models/ : Contains trained or saved models
- src/ : Core application source code
    - api/ : Handles API endpoints and routing
    - chains/ : Implements LangChain pipelines
    - chatbot/ : Core chatbot logic and interaction handling
    - config/ : Configuration settings
    - embeddings/ : Embedding generation logic
    - ingestion/ : Data ingestion pipeline
    - loaders/ : Custom data loaders
    - preprocessing/ : Data cleaning and preprocessing
    - vector_store/ : Vector database handling
- eval.py : Model evaluation script
- train.py : Training pipeline
- preprocess.py : Data preprocessing script
- params.yaml : Configuration parameters
- requirements.txt : Project dependencies
- README.md : Project documentation